Blocking Brute Force Attacks

From OWASP

- 1 Blocking Brute Force Attacks
- 2 Locking Accounts
- 3 Device Cookies
- 4 Finding Other Countermeasures
- 5 Sidebar: Using CAPTCHAS
  - 5.1 Figure 1: Password Authentication Delay: C#
  - 5.2 Figure 2: Password Authentication Delay: VB.NET

Blocking Brute Force Attacks

A common threat web developers face is a password-guessing attack known as a brute force attack. A brute-force attack is an attempt to discover a password by systematically trying every possible combination of letters, numbers, and symbols until you discover the one correct combination that works. If your web site requires user authentication, you are a good target for a brute-force attack.

An attacker can always discover a password through a brute-force attack, but the downside is that it could take years to find it. Depending on the password's length and complexity, there could be trillions of possible combinations. To speed things up a bit, a brute-force attack could start with dictionary words or slightly modified dictionary words because most people will use those rather than a completely random password. These attacks are called dictionary attacks or hybrid brute-force attacks. Brute-force attacks put user accounts at risk and flood your site with unnecessary traffic.

Hackers launch brute-force attacks using widely available tools that utilize wordlists and smart rulesets to intelligently and automatically guess user passwords. Although such attacks are easy to detect, they are not so easy to prevent. For example, many HTTP brute-force tools can relay requests through a list of open proxy servers. Since each request appears to come from a different IP address, you cannot block these attacks simply by blocking the IP address. To further complicate things, some tools try a different username and password on each attempt, so you cannot lock out a single account for failed password attempts.

Locking Accounts

The most obvious way to block brute-force attacks is to simply lock out accounts after a defined number of incorrect password attempts. Account lockouts can last a specific duration, such as one hour, or the accounts could remain locked until manually unlocked by an administrator. However, account lockout is not always the best solution, because someone could easily abuse the security measure and lock out hundreds of user accounts. In fact, some Web sites experience so many attacks that they are unable to enforce a lockout policy because they would constantly be unlocking customer accounts.

The problems with account lockouts are:

- An attacker can cause a denial of service (DoS) by locking out large numbers of accounts.
- Because you cannot lock out an account that does not exist, only valid account names will lock. An attacker could use this fact to harvest usernames from the site, depending on the error responses.
- An attacker can cause a diversion by locking out many accounts and flooding the help desk with support calls.
- An attacker can continuously lock out the same account, even seconds after an administrator unlocks it, effectively disabling the account.

- Account lockout is ineffective against slow attacks that try only a few passwords every hour.
- Account lockout is ineffective against attacks that try one password against a large list of usernames.
- Account lockout is ineffective if the attacker is using a username/password combo list and guesses correctly on the first couple of attempts.
- Powerful accounts such as administrator accounts often bypass lockout policy, but these are the most desirable accounts to attack. Some systems lock out administrator accounts only on network-based logins.
- Even once you lock out an account, the attack may continue, consuming valuable human and computer resources.

Account lockout is sometimes effective, but only in controlled environments or in cases where the risk is so great that even continuous DoS attacks are preferable to account compromise. In most cases, however, account lockout is insufficient for stopping brute-force attacks. Consider, for example, an auction site on which several bidders are fighting over the same item. If the auction Web site enforced account lockouts, one bidder could simply lock the others' accounts in the last minute of the auction, preventing them from submitting any winning bids. An attacker could use the same technique to block critical financial transactions or e-mail communications.

Device Cookies

You may also consider locking out authentication attempts from known and unknown browsers or devices separately. The Slow Down Online Guessing Attacks with Device Cookies article proposes protocol for lockout mechanism based on information about if specific browser have been already used for successful login. The protocol is less susceptible to DoS attacks than plain account locking out and yet effective and easy to implement.

Finding Other Countermeasures

As described, account lockouts are usually not a practical solution, but there are other tricks to deal with brute force attacks. First, since the success of the attack is dependent on time, an easy solution is to inject random pauses when checking a password. Adding even a few seconds' pause can greatly slow a brute-force attack but will not bother most legitimate users as they log in to their accounts.

Note that although adding a delay could slow a single-threaded attack, it is less effective if the attacker sends multiple simultaneous authentication requests.

Another solution is to lock out an IP address with multiple failed logins. The problem with this solution is that you could inadvertently block large groups of users by blocking a proxy server used by an ISP or large company. Another problem is that many tools utilize proxy lists and send only a few requests from each IP address before moving on to the next. Using widely available open proxy lists at Web sites such as http://tools.rosinstrument.com/proxy/, an attacker could easily circumvent any IP blocking mechanism. Because most sites do not block after just one failed password, an attacker can use two or three attempts per proxy. An attacker with a list of 1,000 proxies can attempt 2,000 or 3,000 passwords without being blocked. Nevertheless, despite this method's weaknesses, Web sites that experience high numbers of attacks-adult Web sites in particular-do choose to block proxy IP addresses.

One simple yet surprisingly effective solution is to design your Web site not to use predictable behavior for failed passwords. For example, most Web sites return an "HTTP 401 error" code with a password failure, although some web sites instead return an "HTTP 200 SUCCESS" code but direct the user to a page explaining the failed password attempt. This fools some automated systems, but it is also easy to circumvent. A better solution might be to vary the behavior enough to eventually discourage all but the most dedicated hackers. You could, for example, use different error messages each time or sometimes let a user through to a page and then prompt him again for a password.

Some automated brute-force tools allow the attacker to set certain trigger strings to look for that indicate a failed password attempt. For example, if the resulting page contains the phrase "Bad username or password," the tool would know the credentials failed and would try the next in the list. A simple way to fool these tools is to include also those phrases as comments in the HTML source of the page they get when they successfully authenticate.

After one or two failed login attempts, you may want to prompt the user not only for the username and password but also to answer a secret question. This not only causes problems with automated attacks, it prevents an attacker from gaining access, even if they do get the username and password correct. You could also detect high numbers of attacks system-wide and under those conditions prompt all users for the answer to their secret questions.

Other techniques you might want to consider are:

- For advanced users who want to protect their accounts from attack, give them the option to allow login only from certain IP addresses.
- Assign unique login URLs to blocks of users so that not all users can access the site from the same URL.
- Use a CAPTCHA to prevent automated attacks (see the sidebar "Using CAPTCHAs").
- Instead of completely locking out an account, place it in a lockdown mode with limited capabilities.

Attackers can often circumvent many of these techniques by themselves, but by combining several techniques, you can significantly limit brute-force attacks. It might be difficult to stop an attacker who is determined to obtain a password specifically from your web site, but these techniques certainly can be effective against many attacks, including those from novice hackers. These techniques also require more work on the attacker's part, which gives you more opportunity to detect the attack and maybe even identify the attacker.

Although brute-force attacks are difficult to stop completely, they are easy to detect because each failed login attempt records an HTTP 401 status code in your Web server logs. It is important to monitor your log files for brute-force attacks-in particular, the intermingled 200 status codes that mean the attacker found a valid password.

Here are conditions that could indicate a brute-force attack or other account abuse:

- Many failed logins from the same IP address
- Logins with multiple usernames from the same IP address
- Logins for a single account coming from many different IP addresses
- Excessive usage and bandwidth consumption from a single use
- Failed login attempts from alphabetically sequential usernames or passwords
- Logins with a referring URL of someone's mail or IRC client
- Referring URLs that contain the username and password in the format http://user:password@www.example.com/login.htm
- If protecting an adult Web site, referring URLs of known password-sharing sites
- Logins with suspicious passwords hackers commonly use, such as ownsyou (ownzyou), washere (wazhere), zealots, hacksyou, and the like

Brute force attacks are surprisingly difficult to stop completely, but with careful design and multiple countermeasures, you can limit your exposure to these attacks. Ultimately, the only best defense is to make sure that users follow basic rules for strong passwords: use long unpredictable passwords, avoid dictionary words, avoid reusing passwords, and change passwords regularly.

Sidebar: Using CAPTCHAS

A completely automated public Turing test to tell computers and humans apart, or CAPTCHA, is a program that allows you to distinguish between humans and computers. First widely used by Alta Vista to prevent automated search submissions, CAPTCHAs are particularly effective in stopping any kind of automated abuse, including brute-force attacks. They work by presenting some test that is easy for humans to pass but difficult for computers to pass; therefore, they can conclude with some certainty whether there is a human on the other end.

For a CAPTCHA to be effective, humans must be able to answer the test correctly as close to 100 percent of the time as possible. Computers must fail as close to 100 percent of the time as possible. Ez-gimpy (www.captcha.net/cgi-bin/ez-gimpy), perhaps the most commonly used CAPTCHA, presents the user with an obscured word that the user must type to pass the test. But researchers have since written pattern recognition programs that solve ez-gimpy with 92 percent accuracy. Although these researchers have not made their programs public, all it takes is one person to do so to make ez-gimpy mostly ineffective. Researchers at Carnegie Mellon's School of Computer Science continually work to improve and introduce new CAPTCHAs (see www.captcha.net/captchas).

If you are developing your own CAPTCHA, keep in mind that it is not how hard the question is that matters-it is how likely it is that a computer will get the correct answer. I once saw a CAPTCHA that presents the user with a picture of three zebras, with a multiple-choice question asking how many zebras were in the picture. To answer the question, you click one of three buttons. Although it would be very difficult for a computer program to both understand the question and interpret the picture, the program could just randomly guess any answer and get it correct a third of the time. Although this might seem a satisfactory level of risk, it is by no means an effective CAPTCHA. If you run a free e-mail service and use a CAPTCHA such as this to prevent spammers from creating accounts in bulk, all they have to do is write a script to automatically create 1,000 accounts and expect on average that 333 of those attempts will be successful.

Nevertheless, a simple CAPTCHA may still be effective against brute-force attacks. When you combine the chance of an attacker sending a correct username and password guess with the chance of guessing the CAPTCHA correctly, combined with other techniques described in this chapter, even a simple CAPTCHA could prove effective.

Figure 1: Password Authentication Delay: C#

```
private void AuthenticateRequest(object obj, EventArgs ea)
 {
   HttpApplication objApp = (HttpApplication) obj;
   HttpContext objContext = (HttpContext) objApp.Context;
   // If user identity is not blank, pause for a random amount of time
   if ( objApp.User.Identity.Name != "")
     {
       Random rand = new Random();
       Thread.Sleep(rand.Next(minSeconds, maxSeconds) * 1000);
     }
 }
```

Figure 2: Password Authentication Delay: VB.NET

```
Public Sub AuthenticateRequest(ByVal obj As Object, ByVal ea As System.EventArgs)
 Dim objApp As HttpApplication
 Dim objContext As HttpContext
 Dim ran As Random
 objApp = obj
 objContext = objApp.Context

 ' If user identity is not blank, pause for a random amount of time
 If objApp.User.Identity.Name <> "" Then
   ran = New Random
   Thread.Sleep(ran.Next(ran.Next(minSeconds, maxSeconds) * 1000))
 End If
End Sub
```

Retrieved from "https://www.owasp.org/index.php?title=Blocking_Brute_Force_Attacks&oldid=232111"

Categories: OWASP Columns | Control

- This page was last modified on 9 August 2017, at 03:43.
- Content is available under Creative Commons Attribution-ShareAlike unless otherwise noted.

- Open Web Application Security Project, OWASP, Global AppSec, AppSec Days, AppSec California, SnowFROC, LASCON, and the OWASP logo are trademarks of the OWASP Foundation.